

CIC0198 - TÉCNICAS DE PROGRAMAÇÃO 2

Controle de Versão - GIT

Daniel Porto – daniel.porto@unb.br
Departamento de Ciência da Computação – CIC
Universidade de Brasília – UnB



UnB | IE | CIC

Controle de Versão - GIT

Controle de versão

Conceitos

Comandos básicos

Ramificações

Boas práticas

ALGUÉM JÁ VIU?

"FINAL".doc



FINAL.doc!



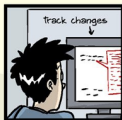
FINAL_rev.2.doc



FINAL_rev.6.COMMENTS.doc



FINAL_rev.8.comments5.
CORRECTIONS.doc



FINAL_rev.18.comments7.
corrections9.MORE.30.doc

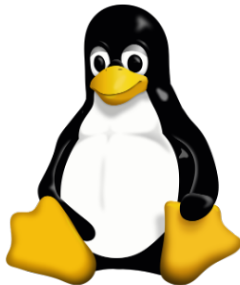


FINAL_rev.22.comments49.
corrections.10.#@\$%WHYDID
ICOMETOGRADSCHOOL????.doc

JORGE CHAN © 2012

O QUE É GIT-SCM

- ▶ Sistema de Controle de Versão Distribuído
- ▶ SCM = Source Control Management
- ▶ Criado por Linus Torvalds (2005)
- ▶ Auxiliar no Desenvolvimento do Linux



GERÊNCIA DE CONFIGURAÇÃO DE SOFTWARE

Durante o desenvolvimento do software queremos saber:

- ▶ O que mudou e quando?
- ▶ Por que mudou?
- ▶ Quem fez a mudança?
- ▶ Podemos reproduzir esta mudança?

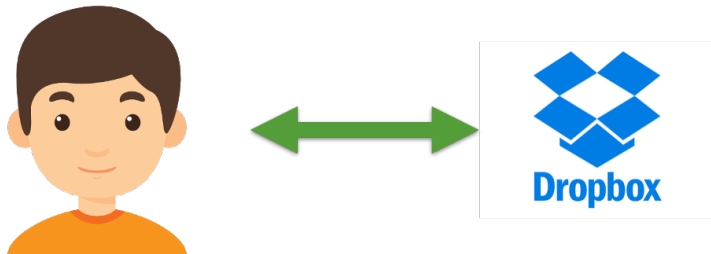
PROBLEMA:

Você precisa editar um código que está no seu Dropbox

Você faz o download do arquivo

Faz as alterações necessárias

Salva o arquivo no Dropbox



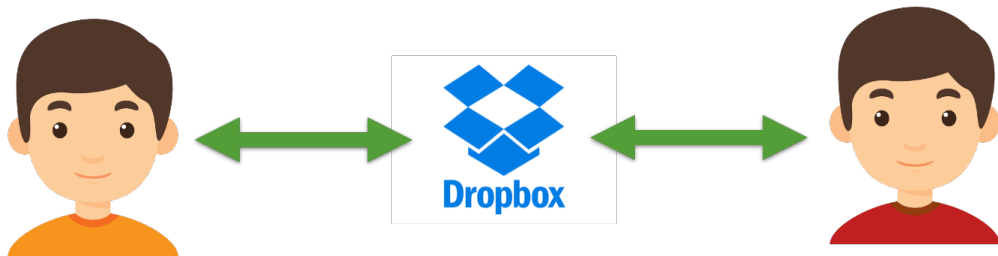
PROBLEMA:

Seu colega também quer editar o código

Vocês baixam o mesmo arquivo juntos

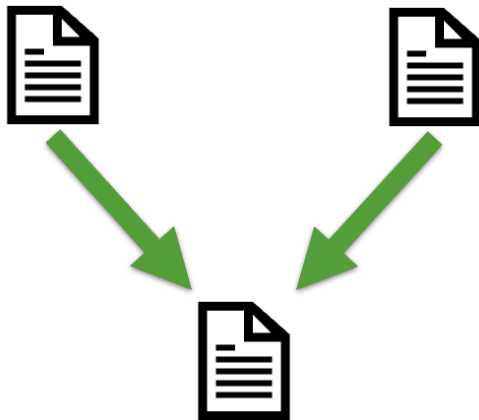
Você edita e salva no Dropbox

Seu colega edita e salva no Dropbox e acaba sobrescrevendo seu código



CONTROLE DE VERSÃO RESOLVE

Controle de versão faz o 'merge' entre as alterações



FERRAMENTAS DE CONTROLE DE VERSÃO

Git

Subversion (SVN)

Mercurial

CVS – Concurrent Versioning System

FERRAMENTAS DE CONTROLE DE VERSÃO

Git

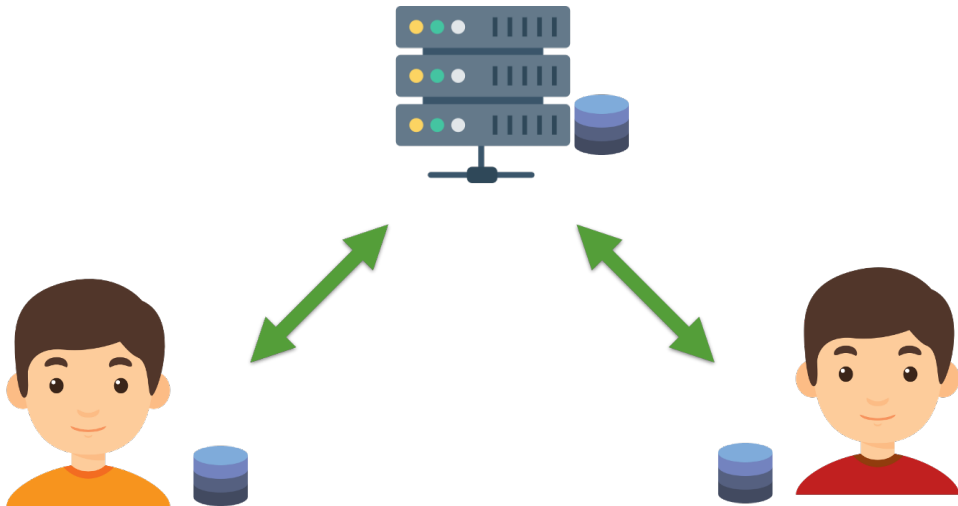
Subversion (SVN)

Mercurial

CVS – Concurrent Versioning System

Git é o mais rápido e eficiente

GIT É DISTRIBUÍDO



CONTROLE DE VERSÃO



O QUE É GITHUB?

Servidor de repositórios Git
Surgiu em 2008
+10 milhões de repositórios
+10 milhões de usuários



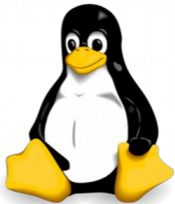
ESTÃO NO GITHUB



Spotify®



USAM GIT



Controle de Versão - GIT

Controle de versão

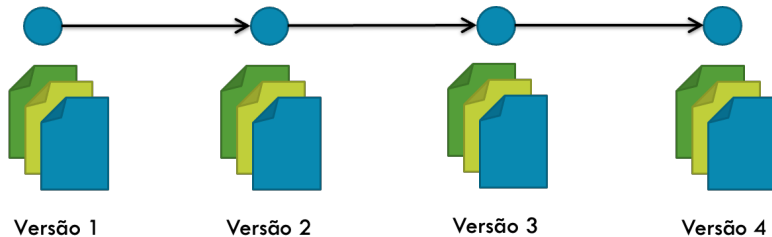
Conceitos

Comandos básicos

Ramificações

Boas práticas

CONCEITOS



Cada versão é uma “foto” do diretório de trabalho e será representado por um círculo azul denominado commit.

CONCEITOS



CONCEITOS



Início



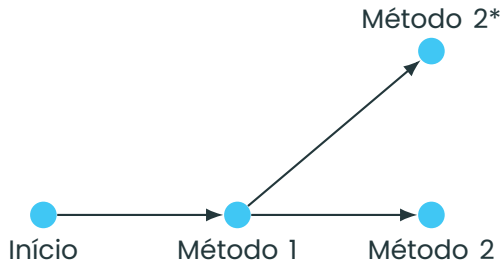
commit

CONCEITOS




commit

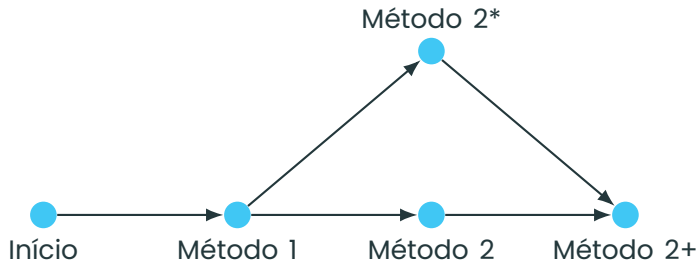
CONCEITOS




commit


branch

CONCEITOS

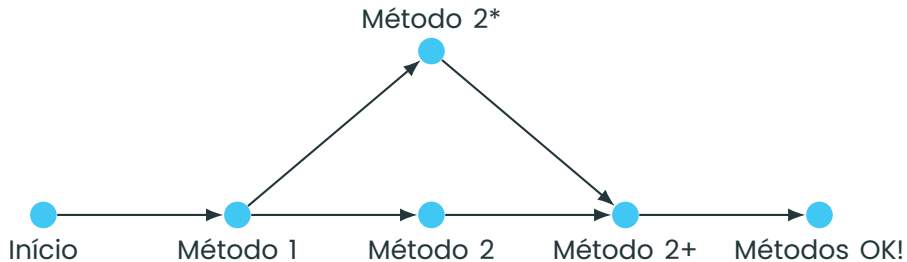



commit


branch


merge

CONCEITOS




commit


branch


merge

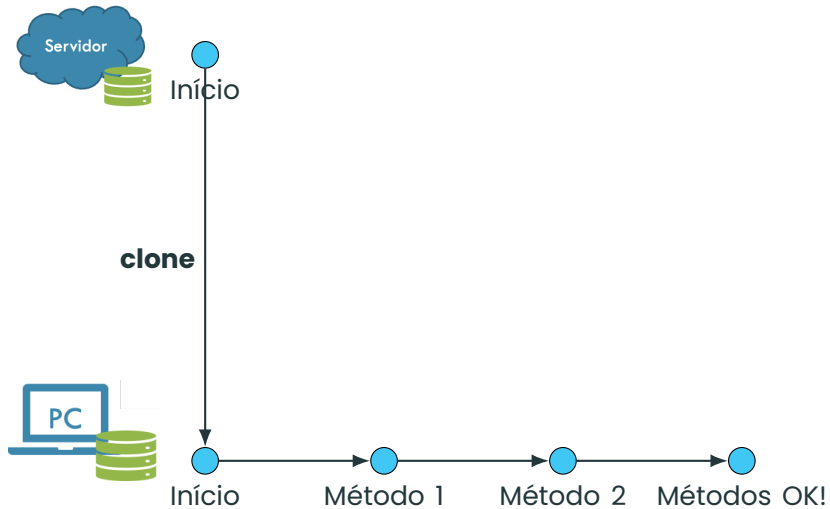
CONCEITOS



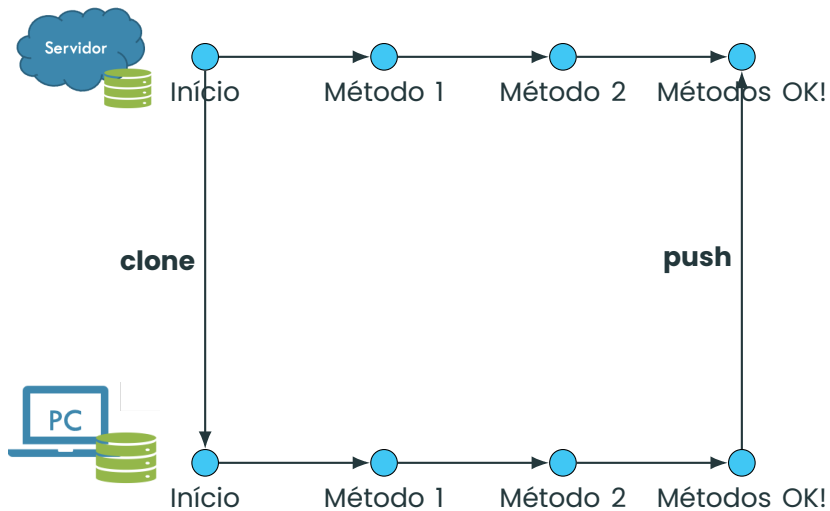
CONCEITOS



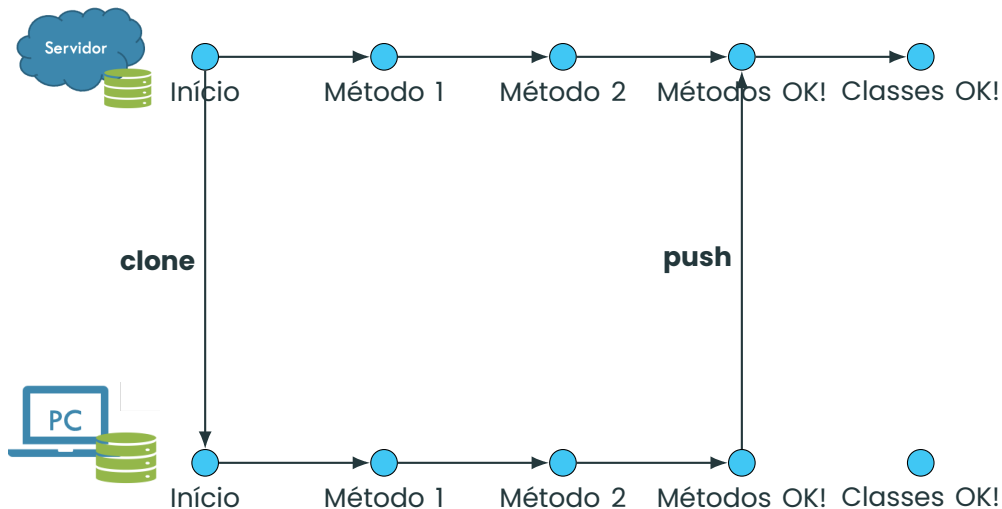
CONCEITOS



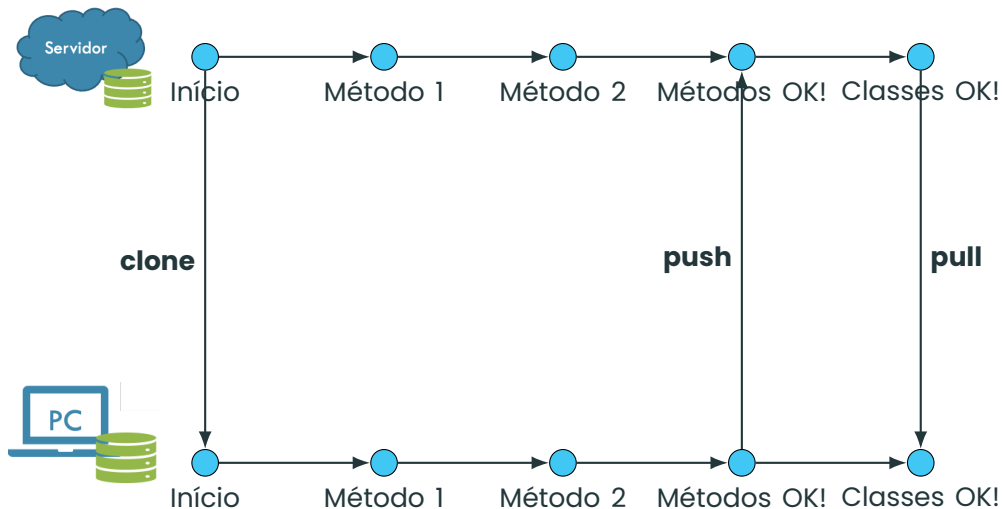
CONCEITOS



CONCEITOS



CONCEITOS



SERVIDORES PARA HOSPEDAGEM



Google Cloud Platform



Controle de Versão - GIT

Controle de versão

Conceitos

Comandos básicos

Ramificações

Boas práticas

INSTALANDO

Windows

- ▶ <https://git-scm.com/download/win>

Linux

- ▶ `sudo apt-get install git`

Mac

- ▶ Instale o Xcode Command Line Tools

INSTALANDO

Windows

- ▶ <https://git-scm.com/download/win>

Linux

- ▶ `sudo apt-get install git`

Mac

- ▶ Instale o Xcode Command Line Tools

Escolha sua interface gráfica

<https://git-scm.com/downloads/guis>



CONFIGURAÇÃO INICIAL DO GIT

```
$ git config --global user.name <nome>
```

Atribui ao nome do usuário.

```
$ git config --global user.email <email>
```

Atribui ao e-mail do usuário.

```
$ git config --global core.editor <editor>
```

Atribui como editor padrão. Ex.: notepad, emacs ...

CRIANDO UM REPOSITÓRIO

```
$ git init
```

Transforma a diretório atual em um repositório git, criando o subdiretório “.git”.

```
$ git init <dir>
```

Cria o diretório <dir> e transforma em um repositório git.

CLONANDO UM REPOSITÓRIO

```
$ git clone <repo>
```

Clona o repositório <repo> para a máquina local.

```
$ git clone <repo> <dir>
```

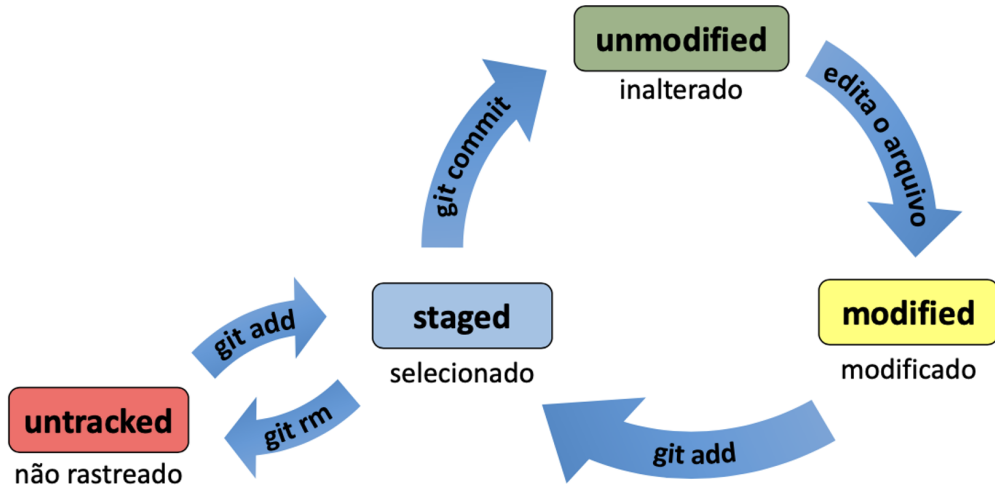
Clona o repositório <repo> para o diretório <dir>.

Ex:

```
$ git clone git@github.com:user/Project.git
```

```
$ git clone https://github.com/user/Project.git
```

TIPOS DE ESTADO DE UM ARQUIVO



.GITIGNORE

Arquivo que contem os arquivos que não serão visíveis pelo git.

Ex:

```
1 Thumbs.db      #Arquivo específico
2 *.html         #Arquivos que terminam com ".html"
3 !index.html    #Exceção, esse arquivo será visível ao git
4 log/           #Diretório específico
5 **/tmp         #Qualquer diretório nomeado de "tmp"
```

PREPARANDO PARA SALVAR ALTERAÇÕES

```
$ git add <arquivo|dir>
```

Adiciona as mudanças do arquivo <arquivo> ou do diretório <dir> para o próximo commit. O arquivo passa a ser rastreado.

```
$ git reset <arquivo>
```

Remove as mudanças do arquivo <arquivo> para o próximo commit.

```
$ git rm --cached <arquivo>
```

Para de rastrear o arquivo <arquivo>.

SALVANDO AS ALTERAÇÕES

```
$ git commit
```

Realiza o commit e abre o editor para inserir uma mensagem.

```
$ git commit -a
```

Adiciona as mudanças dos arquivos já rastreados e realiza o commit. O editor será aberto.

```
$ git commit -m "<msg>"
```

Realiza o commit, com a mensagem <msg>.

SALVANDO AS ALTERAÇÕES

```
$ git commit -am "<msg>"
```

Adiciona as mudanças dos arquivos já rastreados e realiza o commit com a mensagem <msg>.

```
$ git commit --amend -m "<msg>"
```

Substitui o último commit e altera a mensagem para <msg>.

ANALISANDO OS ARQUIVOS DA ÁREA TRANSITÓRIA

```
$ git status
```

Lista os arquivos que estão e que não estão na área transitória, e os arquivos que não estão sendo rastreados.

```
$ git status -s
```

Lista os arquivos de uma forma simplificada.

TAGGING

```
$ git tag
```

Lista as tags existentes.

```
$ git tag -l <tag>
```

Procura pela tag <tag>.

Ex:

```
$ git tag -l 'v.0.*'
```

TAGGING

```
$ git tag <tag> [<commit>]
```

Cria a tag <tag> para o último commit ou para o commit <commit>.

```
$ git tag -a <tag>
```

Cria a tag <tag> completa para o último commit e abre o editor para inserir uma mensagem.

```
$ git tag -a <tag> -m "<msg>"
```

Cria a tag <tag> completa para o último commit com a mensagem <msg>.

ANALISANDO COMMITS

```
$ git show
```

Exibe o último commit.

```
$ git show <commit>
```

Exibe o commit referenciado por <commit>.

```
$ git show <commit>:<arquivo>
```

Exibe o arquivo <arquivo> no commit <commit>.

ANALISANDO UM ARQUIVO

```
$ git blame <arquivo>
```

Exibe quem modificou cada linha do arquivo <arquivo>, incluindo data e commit.

```
$ git blame -L <n>,<m> <arquivo>
```

Exibe quem modificou as linhas de <n> a <m> do arquivo <arquivo>, incluindo data e commit.

DIFERENÇA ENTRE COMMITS

```
$ git diff <commit>
```

Exibe a diferença nos arquivos entre o commit <commit> e o diretório de trabalho.

```
$ git diff --cached <commit>
```

Exibe a diferença nos arquivos entre o commit <commit> e a área transitória.

Controle de Versão - GIT

Controle de versão

Conceitos

Comandos básicos

Ramificações

Boas práticas

CRIANDO RAMIFICAÇÕES

```
$ git branch [-a]
```

Exibe os branches existentes. Na forma completa, exibe também os branches remotos.

```
$ git branch <branch> [<base>]
```

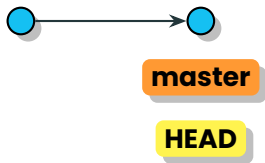
Cria o branch <branch> a partir do commit <base>.

```
$ git checkout -b <branch>
```

Cria o branch <branch> e altera para ele.

CRIANDO RAMIFICAÇÕES

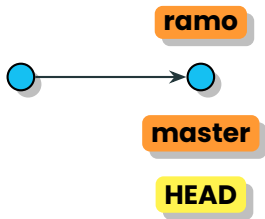
HEAD: aponta para o branch atual.



CRIANDO RAMIFICAÇÕES

```
$ git branch ramo
```

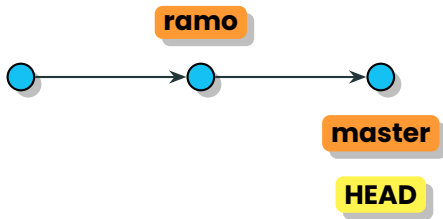
Cria o branch ramo.



CRIANDO RAMIFICAÇÕES

```
$ git commit -a
```

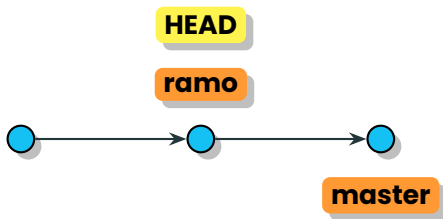
Realiza um commit no branch master.



CRIANDO RAMIFICAÇÕES

```
$ git checkout ramo
```

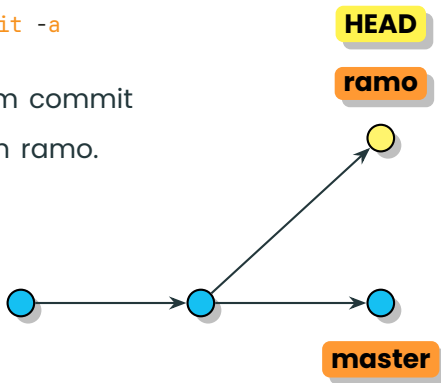
Alterna para o branch ramo.



CRIANDO RAMIFICAÇÕES

```
$ git commit -a
```

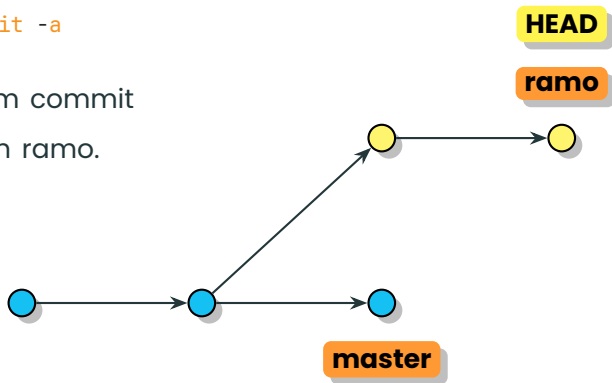
Realiza um commit
no branch ramo.



CRIANDO RAMIFICAÇÕES

```
$ git commit -a
```

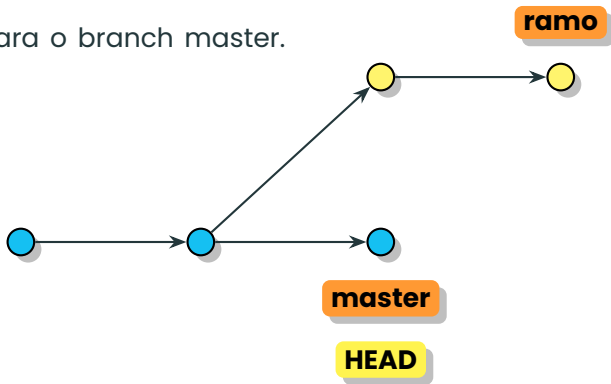
Realiza um commit
no branch ramo.



CRIANDO RAMIFICAÇÕES

```
$ git checkout master
```

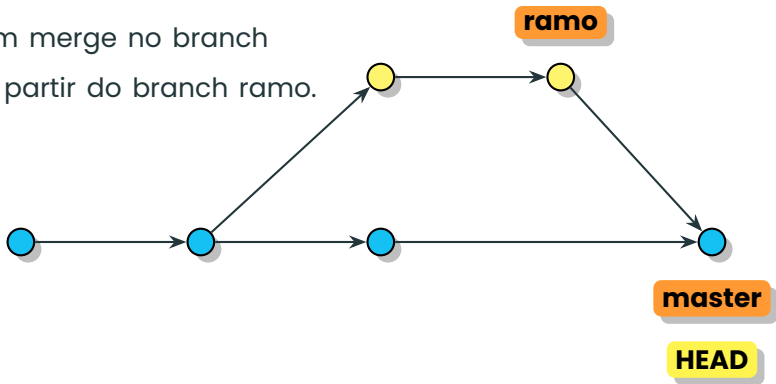
Alterna para o branch master.



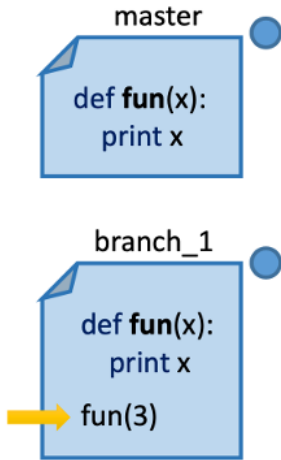
CRIANDO RAMIFICAÇÕES

```
$ git merge ramo
```

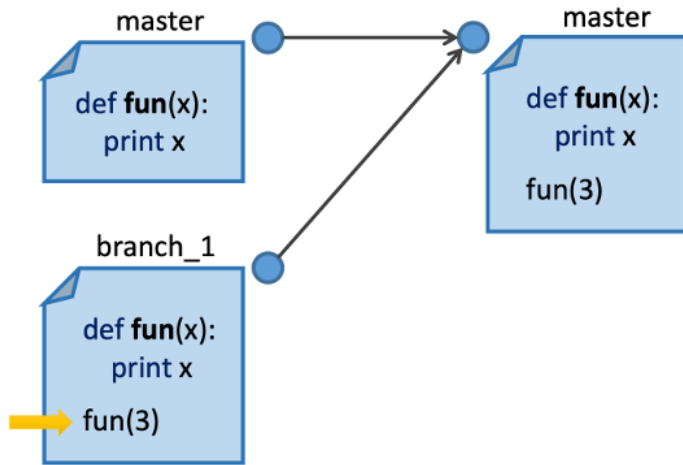
Realiza um merge no branch master a partir do branch ramo.



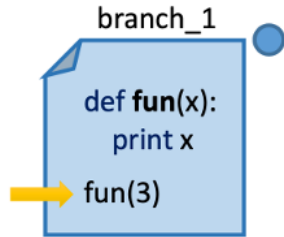
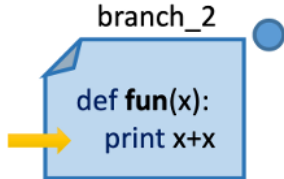
MERGE



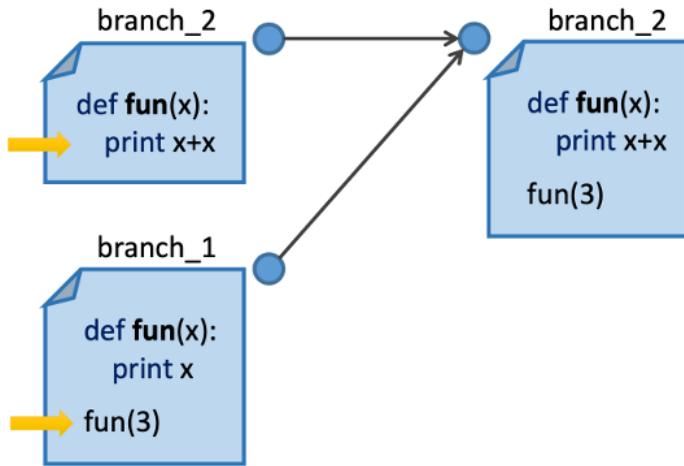
MERGE



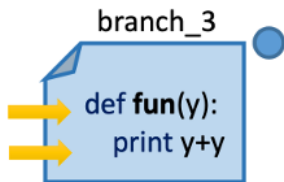
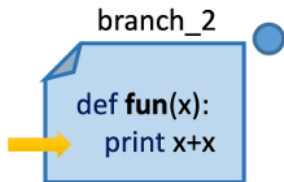
MERGE



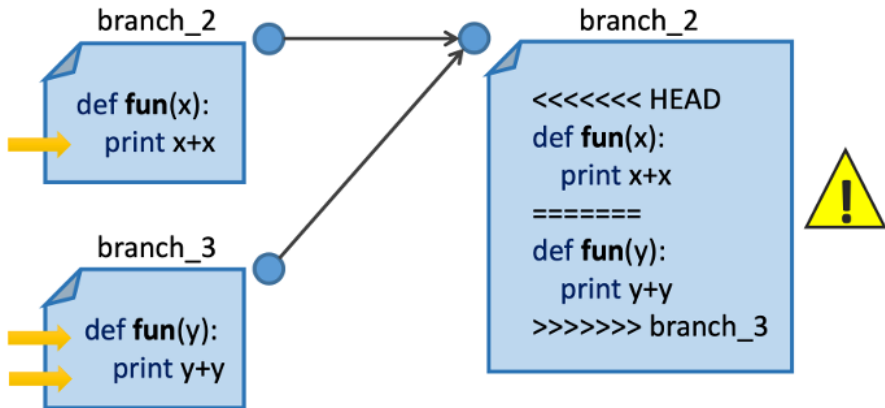
MERGE



MERGE



MERGE



RESOLVENDO CONFLITOS

Alterar o arquivo manualmente

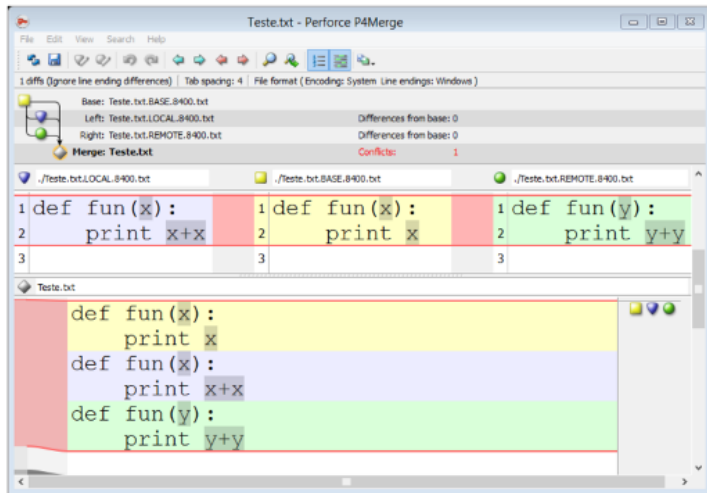
Utilizar uma interface gráfica

- ▶ Ex: kdiff3, tkdiff, meld, xxdiff, vimdiff, p4merge

Com o p4merge configurado, basta fazer:

```
$ git mergetool
```


RESOLVENDO CONFLITOS



The screenshot shows the Perforce P4Merge application window titled "Teste.txt - Perforce P4Merge". The interface includes a menu bar (File, Edit, View, Search, Help) and a toolbar with various icons. Below the toolbar, it displays "1 diffs (ignore line ending differences)", "Tab spacing: 4", and "File format (Encoding: System, Line endings: Windows)".

A diagram on the left shows the merge process: Base (Teste.txt.BASE.8400.txt) is merged with Left (Teste.txt.LOCAL.8400.txt) and Right (Teste.txt.REMOTE.8400.txt) to produce the Merge (Teste.txt). It indicates "Differences from base: 0" for both Left and Right, and "Conflicts: 1" for the Merge.

The main area shows three panels for comparison:

- Left Panel (Teste.txt.LOCAL.8400.txt):**

```
1 def fun(x):  
2   print x+x  
3
```
- Base Panel (Teste.txt.BASE.8400.txt):**

```
1 def fun(x):  
2   print x  
3
```
- Right Panel (Teste.txt.REMOTE.8400.txt):**

```
1 def fun(y):  
2   print y+y  
3
```

The bottom panel shows the merged result in "Teste.txt", with conflicts highlighted in red and the merged content in yellow, light blue, and light green:

```
def fun(x):  
    print x  
def fun(x):  
    print x+x  
def fun(y):  
    print y+y
```

Controle de Versão - GIT

Controle de versão

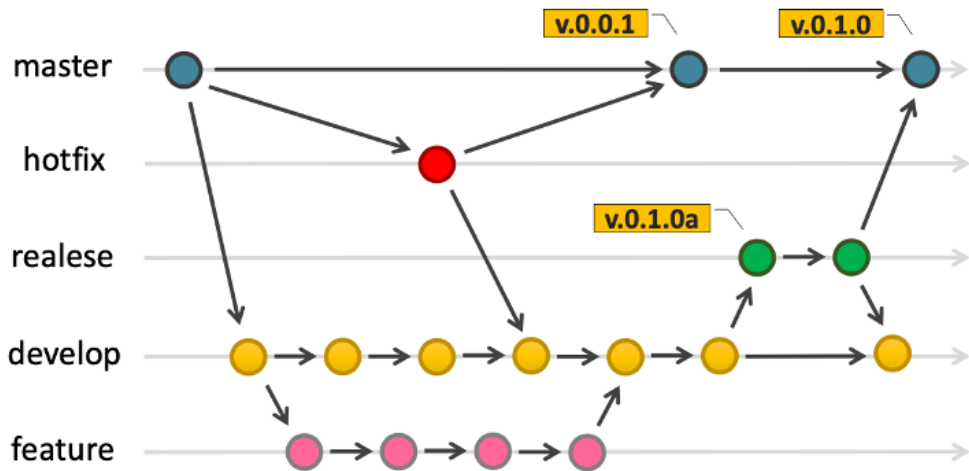
Conceitos

Comandos básicos

Ramificações

Boas práticas

RAMOS / BRANCHES



RAMOS / BRANCHES

master: Versões estáveis.

hotfix: Correção de bugs da versão estável.

release: Teste e correções de versões.

develop: Desenvolvimento.

feature: Implementação de funcionalidades.

NÚMEROS DO VERSIONAMENTO

v[major].[minor].[patch]

NÚMEROS DO VERSIONAMENTO

v[major].[minor].[patch]

[patch]: correção de bugs.

NÚMEROS DO VERSIONAMENTO

v[major].[minor].[patch]

[patch]: correção de bugs.

[minor]: incrementos de funcionalidades compatíveis com versões anteriores.

NÚMEROS DO VERSIONAMENTO

v[major].[minor].[patch]

[patch]: correção de bugs.

[minor]: incrementos de funcionalidades compatíveis com versões anteriores.

[major]: incrementos de funcionalidades incompatíveis com versões anteriores.

NÚMEROS DO VERSIONAMENTO

v[major].[minor].[patch]

[patch]: correção de bugs.

[minor]: incrementos de funcionalidades compatíveis com versões anteriores.

[major]: incrementos de funcionalidades incompatíveis com versões anteriores.

Versões de teste: alpha (a), beta (b)

► Ex: v0.1.9 < v0.1.10 < v0.2.0a < v0.2.0b < v0.2.0

CONVENTIONAL COMMITS

Conventional Commits é uma convenção simples para utilizar nas mensagens de commit.

Ela define um conjunto de regras para criar um histórico de commit explícito.

A mensagem do commit deve ser estruturada da seguinte forma:

```
1 <tipo>[escopo opcional]: <descrição>
2
3 [corpo opcional]
4
5 [rodapé(s) opcional(is)]
```

CONVENTIONAL COMMITS - TIPOS

fix: soluciona um problema no código

- ▶ Ex: Aplicar tratativa para uma função que não está tendo o comportamento esperado e retornando erro.

feat: inclui um novo recurso no código

- ▶ Ex: Acréscimo de um serviço, funcionalidade, endpoint, etc.

build: Alterações que afetam o sistema de compilação ou dependências externas

- ▶ Ex: Mudanças de pacotes npm, etc.

CONVENTIONAL COMMITS

ci: Alterações em nossos arquivos e scripts de configuração de Integração Contínua

- ▶ Ex: Jenkins, Circle, Travis, BrowserStack, etc.

docs: Apenas alterações na documentação

- ▶ Ex: adicionar informações na documentação da API, mudar o README, etc.

perf: Uma alteração no código que melhora o desempenho

- ▶ Ex: alterar ForEach por while, melhorar a query ao banco, etc.

CONVENTIONAL COMMITS

refactor: Uma alteração no código que não corrige um bug nem adiciona um recurso

- ▶ Ex: Mudanças de código após um code review

style: Alterações que não afetam o significado do código (espaços em branco, formatação, ponto-e-vírgulas ausentes, etc.)

- ▶ Ex: Espaços em branco, formatação, ponto-e-vírgulas ausentes, etc. Não afeta o comportamento da aplicação

chore: Mudanças no projeto que não afetem o sistema ou arquivos de testes. São mudanças de desenvolvimento.

- ▶ Ex: Mudar regras do eslint, adicionar prettier, adicionar mais extensões de arquivos ao .gitignore

CONVENTIONAL COMMITS

test: Adicionar testes ausentes ou corrigir testes existentes

- ▶ Ex: Criação de testes unitários.

revert: indica a reverção de um commit anterior

- ▶ Ex: Alguém subiu uma besteira pra produção e temos que reverter

CONVENTIONAL COMMITS

```
1 <tipo>[escopo opcional]: <descrição>  
2  
3 [corpo opcional]  
4  
5 [rodapé(s) opcional(is)]
```

CONVENTIONAL COMMITS

Escopo (Opcional)

- ▶ O escopo deve ser o nome do pacote afetado.

Descrição

- ▶ A descrição contém uma descrição **sucinta** da alteração

Corpo (Opcional)

- ▶ O corpo deve incluir a motivação para a mudança e contrastá-la com o comportamento anterior.

Rodapé (Opcional)

- ▶ Introduz uma modificação que quebra a compatibilidade da API
- ▶ Uma BREAKING CHANGE (BREAKING CHANGE: ou !) pode fazer parte de commits de qualquer tipo.

CONVENTIONAL COMMITS

Isso tudo evita mensagens do tipo:

CONVENTIONAL COMMITS

Isso tudo evita mensagens do tipo:

```
git commit -m "🙏"
```

LINKS

Mais informações:

<https://www.conventionalcommits.org/pt-br>

Resumo dos comandos git mais usados:

<https://github.com/joshnh/Git-Commands/blob/master/READMEpt.md>



Este documento está licenciado sob a Licença Atribuição-NãoComercial-Compartilhalgual 4.0 Internacional Creative Commons. Para visualizar uma cópia desta licença, visite <http://creativecommons.org/licenses/by-nc-sa/4.0/> ou mande uma carta para Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

